Agentic AI
Learners' Space 2026

Assignment 2
Topics: Retrieval Augmented Generation (RAG) and Fine-Tuning
Submission Deadline: 23rd June 11:59 pm

Hello! We hope you enjoyed the Week 2 resources on Retrieval Augmented Generation (RAG) and Fine-Tuning. This assignment will test your conceptual understanding and practical skills from those materials. Follow through the provided instructions. Good luck!

Instructions:

- The Assignment contains 3 Questions. You can write your Python code in either Jupyter notebook, Pycharm, VS Code, or any other code editor of your choice.
- For submission, either submit the pdf of a single .ipynb (Jupyter Notebook) or submit the pdf file of a word document in which you will copy/paste your python code along with the screenshot of the respective output. Name the final submission pdf file as: AgenticAI_Week2_<YourName>_<RollNo>
- All three questions must be attempted. Code must run end-to-end without errors.
- You can find the text file required for the questions 1,2 and 3 attached on the Agentic AI Notion Page under the Assignment Toggle Heading (for Week 2).
- You are free to use any other input text of your choice as well for these questions (make sure to attach the input text used by you (if different from the provided text) in the final submission pdf)

Question 1: Build a Minimal RAG Pipeline from Scratch
You will implement the core RAG pipeline without using any high-level RAG framework. Everything: chunking, embedding, vector search, and generation must be wired by you.

Input: Load the provided text file as a separate file in your code editor (in the same folder as your main code). Import the text file into your main code. Here's a short guide to refer to opening, reading and writing to a text file in Python.

https://www.geeksforgeeks.org/python/reading-writing-text-files-python/

Part A: Chunking

Implement fixed-size chunking with overlap.

```
# Starter signature — implement this function
def chunk_text(text: str, chunk_size: int = 200, overlap:
int = 40) -> list[str]:
    """
    Split `text` into chunks of ~chunk_size words,
    with `overlap` words shared between consecutive chunks.
    Returns a list of chunk strings.
    """
    ...
```

For output: Call the function and print the total number of chunks to the console.

Part B: Embedding & Vector Store

Use sentence-transformers (model: all-MiniLM-L6-v2) to embed your chunks.
Store embeddings in a numpy array (no vector DB required here, just numpy).
Implement cosine similarity manually (define a function named cosine_similarity which takes a and b as parameters)
Formula: $\cos(\theta) = A \cdot B / \|A\|\|B\|$). Do not use sklearn for this part.
Then: run 3 different queries against your document and print the top-3 retrieved chunks for each. Include the cosine scores alongside.

```
from sentence_transformers import SentenceTransformer
import numpy as np
model = SentenceTransformer('all-MiniLM-L6-v2')
# TODO: embed all chunks → shape (num_chunks, embedding_dim)
chunk_embeddings = ...
# TODO: implement cosine similarity retrieval
def retrieve(query: str, top_k: int = 3) -> list[str]:
    """Return the top_k most similar chunks to `query`."""
```

Part C: Generation

Plug your retriever into an LLM call. Use the google/flan-t5-base via HuggingFace.

Firstly, install the package by running this command in the terminal.

```
pip install transformers torch
```

Then use this code to start off:

```
from transformers import pipeline
generator = pipeline("text2text-generation", model="google/
flan-t5-base")
def rag_answer(query: str) -> str:
    chunks = retrieve(query, top_k=3)
    context = "\n\n".join(chunk for chunk, _ in results)
    prompt = f"""Answer the question using ONLY the context
below.
    If the answer is not in the context, say 'I don't
know'.
    Context:
    {context}
    Question: {query}
    Answer:"""
    # TODO: call your LLM here and return the response
string
    ...
```

if this gives error then try this instead:

```
from transformers import pipeline
generator = pipeline("text-generation", model="google/flan-
t5-base")
def rag_answer(query: str) -> str:
    chunks = retrieve(query, top_k=3)
    context = "\n\n".join(chunk for chunk, _ in results)
    prompt = f"""Answer the question using ONLY the context
below.
    If the answer is not in the context, say 'I don't
know'.
    Context:
    {context}
    Question: {query}
```

```
    Answer:"""
    # TODO: call your LLM here and return the response
string
    ...
```

Run 2 queries: one answerable from your document and one deliberately outside the document's scope. Show that your pipeline returns "I don't know" (or equivalent) for the out-of-scope query. This is the hallucination guard.

Question 2: Chunking Strategy Showdown

Chunking is one of the most underappreciated decisions in RAG. In this question you will empirically verify that claim by comparing three chunking strategies and measuring their effect on retrieval quality.

Input: Use the same document from Q1, or replace it with a longer one (3,000+ words). You need enough content for the differences between strategies to show up.

Part A: Implement Three Chunkers

Implement all three of the following. Each function must return a list of strings.

Hint: import re and numpy

Refer to the Official Python re Documentation for reference:
https://docs.python.org/3/library/re.html

- fixed_chunk(text, size=300, overlap=50). Reuse or refine your Q1 implementation.
- sentence_chunk(text). Split at sentence boundaries (split on '. ', '? ', '! '). Group sentences into chunks of ~5 sentences each with 1-sentence overlap.
- sliding_window_chunk(text, window=400, step=100). Each chunk is 400 words, starting every 100 words (heavy overlap).

For each strategy, print: number of chunks produced, mean chunk length in words, and min/max chunk lengths.

Part B: Retrieval Comparison

Build a separate retriever for each chunking strategy (embed with all-MiniLM-L6-v2 as before). Create a benchmark of 5 question-answer pairs you write manually from your document (you know the correct answers).

For each strategy and each question, check whether the correct answer appears in the top-3 retrieved chunks (string match is fine). Compute Hit Rate = (questions where answer found in top-3) / 5.

Expected output format

Strategy	Chunks	Mean Len	Hit Rate
Fixed-size	42	298 wds	3/5
Sentence-based	38	310 wds	4/5
Sliding window	91	400 wds	4/5

Question 3: RAG vs Fine-Tuning: Decision and Demo

This question tests conceptual depth and your ability to think like a system designer. You'll write a decision framework and build a small demonstration to back it up.

Part A: Build the Decision Tree

The golden rule is: use RAG for new facts, and Fine-Tuning for new behavior. Build a more granular decision tree in code as a function:

```
def recommend_approach(scenario: dict) -> str:
    """
    Input keys:
      - data_changes_frequently: bool
      - need_specific_output_format: bool
      - budget: str  # 'low' | 'medium' | 'high'
      - latency_sensitive: bool
      - knowledge_type: str  # 'factual' | 'behavioral' |
'both'
    Returns one of: 'RAG', 'Fine-Tuning', 'RAG + Fine-
Tuning', 'Prompt Engineering only'
    with a 1-sentence justification.
    """
```

Test it on at least 5 diverse scenarios (you define them). At least one scenario must return 'RAG + Fine-Tuning' and one must return 'Prompt Engineering only'.

Part B: Hallucination Stress Test

Take your RAG pipeline from Q1. Run it on 6 queries, 3 that are answerable from your document, and 3 that are completely outside the document's scope (make them plausible-sounding but factually absent).
For each query, log:

- The top retrieved chunk (first 100 words).
- The cosine similarity score of the top chunk.
- The LLM's final answer.

Plot a bar chart: cosine similarity score for all 6 queries, color-coded (green = answerable, red = out-of-scope). You should observe a pattern. Answerable queries should have higher similarity scores.
Hint: import matplotlib.pyplot as plt
Refer to the matplotlib documentation: https://matplotlib.org/3.5.3/api/_as_gen/matplotlib.pyplot.html
Hint: If your pipeline hallucinated on an out-of-scope query (gave a confident wrong answer), investigate why. Was the cosine score suspiciously high? Was your grounding prompt weak? Include this observation in your analysis.

Resources for this assignment are listed on the Notion page. You are encouraged to re-watch or re-read any of them as needed. Reach out on the Whatsapp group or the MS Teams channel for any doubt.

All the best!